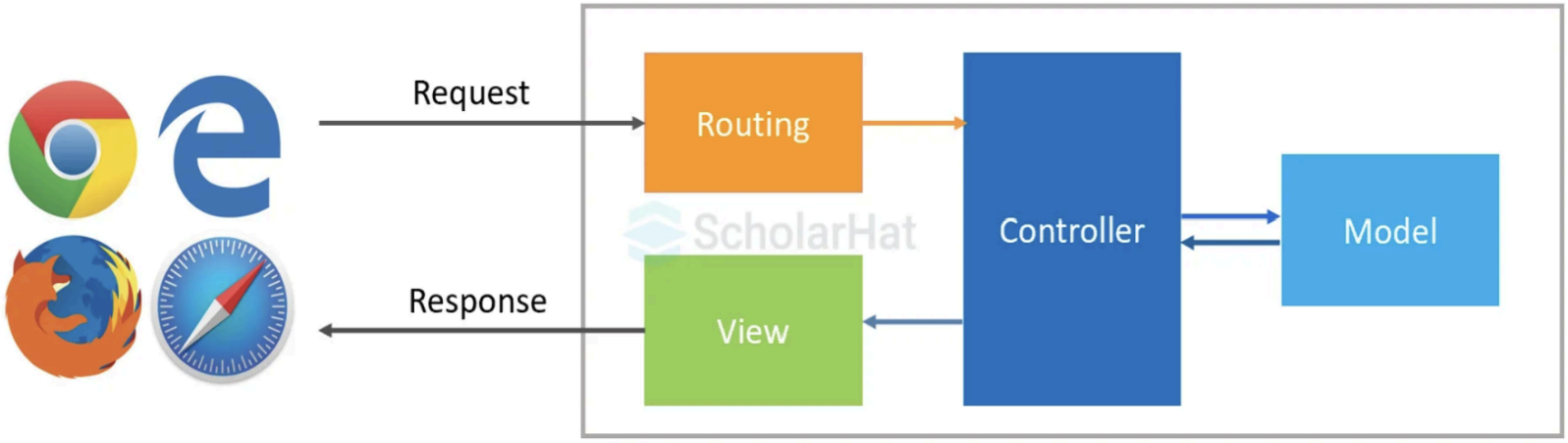
Express Routing

- Express Routing
  - Routing
  - Manejadores de solicitud
  - Buenas prácticas
  - Métodos Http soportados por Express
  - Ejecutar función para todos los métodos `app.all()`
  - Encadenando manejadores de ruta `app.route()`
  - Manejador de ruta independiente `express.Router()`



Routing

El ruteo se refiere a la acción de asociar una solicitud Http con la función que la va a a manejar.
Express nos proporciona una manera de organizar esto.
Cuando se hace una solicitud Http a una aplicación Express
esta solicitud tendrá un método (verbo Http) y una ruta.

`app.VERBO('ruta',función)`
La función `app.verbo()` nos permite ejecutar funciones en esa solicitud.
Es decir: `app.método(ruta,callback1(),callback2()...callback3())`

Para un método GET:

```
app.get('ruta/recurso', function(req,res){
    // Manejamos la ruta
    return next();
})
```

Para un método POST (usando funciones flecha):

```
app.post('ruta/recurso', (req,res)=>{
    // Manejamos la ruta
    next();
})
```

Manejadores de solicitud

Los callbacks que le pasamos a los métodos **`app.get()`** y **`app.post()`** etc...
son los manejadores de la solicitud, ya que toman la solicitud, la procesan y envían una respuesta.

Debido a que podemos encadenar manejadores de solicitud es que se les conoce como middlewares. Estos middlewares aceptan un tercer parámetro **`next`**, si dentro de la función llamamos a **`next()`** la ejecución pasará a la siguiente función middleware.

```
app.post('ruta/recurso', function(req,res,next) {   // Primera función middleware
    return next();                                   // Ejecutar siguiente función middleware
}, function(req,res,next) {                          // Segunda función middleware
    return next();                                   // Ejecutar siguiente función middleware
})
```

Podemos usar **`next('route')`** para saltarnos la cadena de middleware del manejador actual y pasar la ejecución
a un segundo manejador con el mismo método y ruta (útil para pre-procesamiento o separación de lógica).

```
app.get('/recurso1', function(req,res,next) {     // Primera función middleware del bloque 1
    return next('route');                         // Salta las funciones restantes del bloque y pasa al sig. manejador
}, function(req,res,next){                        // Segunda función middleware del bloque 1
    return next();                                // Pasa a la siguiente función middleware
});

app.get('/recurso1', function(req,res,next) {     // Primera función Middleware del bloque 2
    return next();                                // Pasa a la siguiente función
});
```

Buenas prácticas

Para efectos de re-usabilidad del código, podemos asignar las funciones a variables:

```
let validar = function (req, res, next) {
                        // Proceso de validación
    return next();      // Ejecutar siguiente función middleware
}
let procesar = function (req, res, next) {
                        // Proceso de la solicitud
    return next();      // Ejecutar siguiente función middleware
}
let enviarResp = function (req, res) {
                         // Proceso de la solicitud
    res.end();           // Envía respuesta al cliente
}
```

Posteriormente podemos pasarlas a los métodos `app.VERBO()`

```
app.get('/ruta', validar, procesar, enviarResp);
```

También podemos crear un array con las funciones y pasarlo a los métodos `app.VERBO()`

```
let funciones = [validar, procesar, enviarResp];
app.post ('/ruta', funciones);
```

Métodos Http soportados por Express

Los métodos HTTP comúnmente usados en la arquitectura REST son:

- `app.get()`: Obtiene un recurso
- `app.post()`: Envía un recurso para que sea dado de alta
- `app.delete()`: Elimina un recurso
- `app.put()`: Actualiza completamente un recurso
- `app.patch()`: Actualiza parcialmente un recurso
- `app.head()`: Obtiene solamente los encabezados de un recurso
- `app.options()`: Obtiene los métodos del recurso

Adicionalmente a los métodos anteriores Express soporta también los siguientes métodos:

`trace`	`copy`	`lock`	`mkcol`	`move`	`patch`
`purge`	`propfind`	`proppatch`	`unlock`	`report`	`search`
`mkactivity`	`m-search`	`notify`	`subscribe`	`unsubscribe`	`connect`

Ejecutar función para todos los métodos `app.all()`

Cuando queramos ejecutar un manejador de solicitud en una ruta en particular
sin importar el método de la solicitud podemos usar **`app.all()`**

```
app.all('ruta/recurso', function(req,res,next){   // Ejecutamos una función para todos los métodos de la ruta
    return next();                                 // Pasar a la siguiente función middleware
});

app.get('ruta/recurso', function(req,res,next){
    /* Se ejecuta función para el método GET */
})
app.post('ruta/recurso', function(req,res,next){
    /* Se ejecuta función para el método POST */
})
```

Encadenando manejadores de ruta `app.route()`

Al método **`app.route()`** podemos especificarle una ruta de acceso y encadenarle los manejadores de ruta.

```
app.route('/ruta')
    .get((req,res,next)=>{  /* Manejador del método GET  de la ruta */ })
    .post((req,res,next)=>{ /* Manejador del método post de la ruta */ })
    .put((req,res,next)=>{  /* Manejador del método PUTT de la ruta */ })
```

Manejador de ruta independiente `express.Router()`

`express.Router()` nos permite hacer todo el manejo de una ruta en un archivo independiente (un módulo).
Este modulo posteriormente lo podemos importar en nuestra aplicación Express principal.

Módulo independiente que va manejar una ruta (**`recursoRouter.js`**):

```
const express = require('express');
const router  = express.Router();

router.get('/',(req,res,next)=>{  /* Manejador del método get  */ })

router.post('/',(req,res,next)=>{ /* Manejador del método post */ })

router.put('/',(req,res,next)=>{  /* Manejador del método put  */ })

module.exports.router = router;
```

Archivo principal de la aplicación Express (**`index.js`**):

```
const express = require('express');
const recursoRouter = require('./recursoRouter.js');

const app = express();

app.use('/recurso',recursoRouter.router);

app.listen(8082, function(err) {
    console.log("Servidor escuchando en puerto 8082");
});
```